PharmaStock

How it works, layer by layer, and how to deploy it

Medicine stock management for health centers, built on ERPNext Stock and the Frappe framework.
Repository: github.com/Arielpetit/pharmastock · Target versions: Frappe and ERPNext v16 · Document written from the source code (October 2026).

Contents

1. The idea in one page
2. The layers, from the browser to the database
3. Layer 1: the browser app (Vue, PWA, offline)
4. Layer 2: how the page reaches the user (Frappe web layer)
5. Layer 3: the Frappe framework and what we use from it
6. Layer 4: the PharmaStock API (`api.py`)
7. Layer 5: the ERPNext Stock module (what we reuse)
8. Layer 6: database, cache, workers
9. Why PharmaStock has to be installed into ERPNext (and what the "desk" is)
10. End-to-end flows (login, receive, give out, request, dashboard, offline)
11. What happens at install time
12. Deployment: requirements, options, and what we ran
13. Problems we met and the lessons
14. Operations checklist, and a file map

1. The idea in one page

ERPNext is a full business system. Its **Stock** module already knows how to track items, batches with expiry dates, warehouses, receipts, issues and re-order levels. But ERPNext's screens are built for accountants and storekeepers, and are heavy for a pharmacy assistant on a phone.

PharmaStock keeps the **ERPNext Stock engine** and replaces the **screens**. Pharmacy staff use a small, fast, installable web app at `/pharmacy` (English and French, works offline). Every button in that app ends up creating or reading standard ERPNext records. Nothing is duplicated, so a pharmacist's stock-in is the same ERPNext Stock Entry that an administrator would see in the ERPNext desk.

> **Three sentences to remember.** (1) The browser app only talks to our own API, `pharmastock.api.*`. (2) That API runs *inside* the ERPNext site and calls ERPNext's stock document types directly in Python, not over HTTP. (3) ERPNext then does the real stock work: validation, batches, the stock ledger and re-order rules.

2. The layers, from the browser to the database

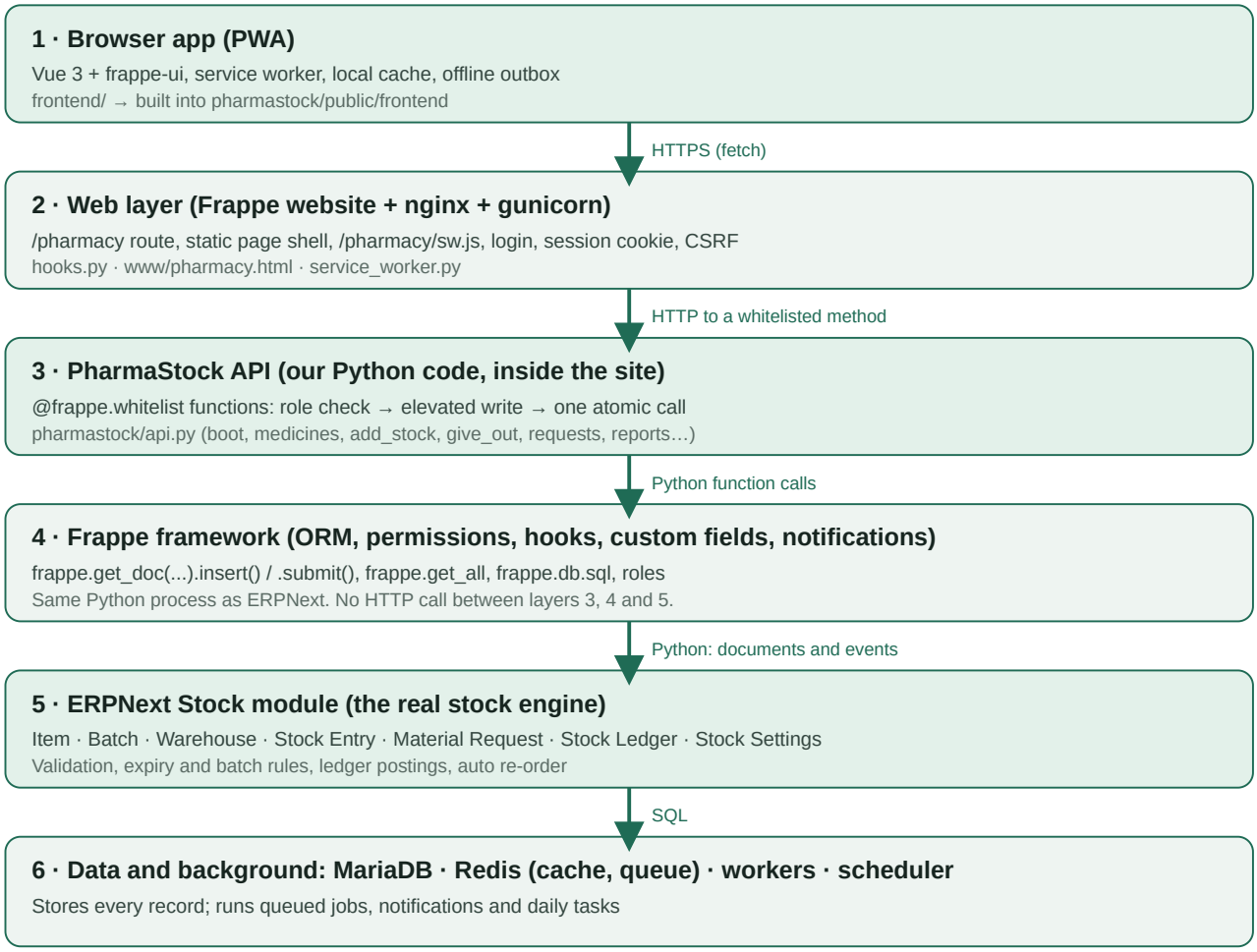


Figure 1. Request path. Only the arrow from layer 1 to layer 2 crosses the network.

3. Layer 1: the browser app

What it is. A single-page app written with **Vue 3** and the **frappe-ui** component library, built with **Vite**. Source is in `frontend/`; the built result is committed into `pharmastock/public/frontend` and `pharmastock/www/pharmacy.html`, so a server does not need Node.

Piece	Where	Job
Pages	`frontend/src/pages/*.vue`	Dashboard, Medicines, Medicine detail, Give out, Requests, Reports, Staff, Notifications, Account, Login
API client	`src/api.js`	The only network code. Calls `/api/method/pharmastock.api.<name>` on the same site
Offline layer	`src/api.js`, `src/pwa.js`	Caches reads in `localStorage`; keeps failed writes in an outbox and retries them
Service worker	built by `vite-plugin-pwa` (Workbox)	Pre-caches the app files so `/pharmacy` opens without internet

Language and theme	`i18n.js`, `fr.js`, `theme.js`	English (in code) and French; dark and light

How offline works, exactly

1. **Reads.** Each successful read is saved in the browser. If the network is down, the app shows the last saved copy and a banner: "You are offline. You see the last saved data…".
2. **Writes.** Every write (add stock, give out, request) is created with a unique `client_id`. If the request cannot be sent, it waits in an **outbox** in the browser.
3. **Sync.** When the connection returns, the outbox is sent again. The server stores the `client_id` on the record and, if it sees the same one twice, returns the first result instead of recording the movement again. That is what guarantees "exactly once".

> **Limit.** Offline mode protects a *user's device* that loses internet. It does not keep the system running if the *server* is down. If the machine hosting ERPNext is off, nobody can sync.

4. Layer 2: how the page reaches the user

This layer is ordinary Frappe web serving. PharmaStock only adds a few lines of configuration in `pharmastock/hooks.py`:

Hook	What it does
`website_route_rules`	Sends `/pharmacy` and every sub-path (`/pharmacy/medicines`, …) to the same page, `www/pharmacy.html`. The Vue router then decides which screen to show. This is why a browser refresh on any screen works.
`www/pharmacy.html` + `pharmacy.py`	A static HTML shell (no user data inside, `no_cache`). User data and a fresh CSRF token arrive afterwards from `pharmastock.api.boot`.
`page_renderer` (`service_worker.py`)	Serves the service worker at `/pharmacy/sw.js`. A service worker can only control pages under the folder it is served from. The build puts it under `/assets/…`, so it is re-served from `/pharmacy/` with the header `Service-Worker-Allowed: /pharmacy`.
`role_home_page`	After a normal Frappe login, users with the Pharmacy roles land on `/pharmacy` instead of the ERPNext desk.
`add_to_apps_screen`	Adds a PharmaStock tile with its logo to the ERPNext apps screen (the page you see after `/desk`), for users who are allowed into the desk.

Static files

The built JavaScript, CSS and images live in `pharmastock/public/frontend` and are served at `/assets/pharmastock/frontend/…`. In production the **nginx** container serves `/assets` itself, so the app's public folder must exist inside that container (our custom Docker image makes sure it does). Python requests go to **gunicorn**, and real-time messages to a **Node websocket** process.

5. Layer 3: the Frappe framework

Frappe is the Python web framework underneath ERPNext. It provides the building blocks. PharmaStock is a **Frappe app**: a Python package with a `hooks.py` that Frappe loads, installed into a **site**.

Frappe feature	How PharmaStock uses it
Apps and sites	A "bench" holds apps (frappe, erpnext, pharmastock). A "site" is one database plus its settings. PharmaStock is installed into the same site as ERPNext, which is why it can reach ERPNext's data directly.
`@frappe.whitelist()`	Turns a Python function into a web endpoint at `/api/method/pharmastock.api.<function>`. This is our entire public API.
Document ORM	`frappe.get_doc({...}).insert()` and `.submit()` create records; `frappe.get_all` reads them; `frappe.db.sql` is used for the ledger reports.
Document events	When a Stock Entry is submitted, ERPNext's own code runs (validation, ledger posting). We never re-implement it.
Roles and permissions	Two custom roles: `Pharmacy Staff` and `Pharmacy Admin`, both with `desk_access = 0`.
Custom Fields	Extra columns added to ERPNext records without editing ERPNext (see below).
Notifications	Three rules created at install: expiring batch (60 days ahead), low-stock re-order created, and "stock request waiting for approval".
Sessions and CSRF	Login uses the standard Frappe session cookie. `boot()` returns a fresh CSRF token so even a cached page can post.
Hooks	`after_install`, `after_migrate`, `setup_wizard_complete` run our setup code at the right moments.

Custom fields added to ERPNext records

On	Fields	Purpose
Item	Dosage form, Strength, Storage location	Pharmacy details of a medicine
Stock Entry	Reason, Recipient, Client ID (hidden)	Why stock left, who got it, and the duplicate-protection key
Material Request	Urgency, Reason, Client ID, Rejected, Decision note	The stock-request workflow

6. Layer 4: the PharmaStock API

`pharmastock/api.py` (about 700 lines) is the heart of the app. Each function follows the same recipe:

1. **Check the role.** `_require_staff()` or `_require_admin()`. A guest or a user without a Pharmacy role gets a permission error.
2. **Elevate for the write.** `@_writes` briefly runs the code as the system user (`_as_system`). Pharmacy users deliberately have no ERPNext permissions, and ERPNext checks permissions deep inside stock posting, so the write is done with full rights *after* our own role check.

3. **Do the work in one call.** One request creates the whole record, so a dropped connection cannot leave half a stock movement.
4. **Restore the real owner.** `_keep_owner` sets the document's owner back to the real user, so history shows who actually did it.
5. **Be repeat-safe.** If `client_id` was already recorded, return the earlier result (`_existing`).

The endpoints

Function	Who	What it does
`boot`	Anyone (guest gets `user: null`)	Current user, role, company, currency, option lists (categories, forms, units, reasons, urgencies), CSRF token
`medicines`, `medicine`	Staff	List with search, category, status and expiry filters; one medicine with stock, batches and recent history
`save_medicine`	Staff	Create or edit an Item; optionally receive its first stock
`add_stock`	Staff	Create a Batch (lot + expiry) and a submitted **Material Receipt** into the Pharmacy warehouse
`give_out`	Staff	Choose batches (`_allocate`) and submit a **Material Issue**. Earliest expiry first; expired batches only when the reason is "Expired stock"
`request_stock`	Staff	Create a Material Request that waits for an administrator
`requests`, `decide_request`	Staff / Admin	List requests; approve (submit) or reject (flag + note)
`movements`, `expiring`, `dashboard`	Staff	Read-only reports from the Stock Ledger and batch data
`notifications`, `mark_notifications_read`	Staff	The bell menu
`staff`, `save_staff`	Admin	Manage pharmacy users and their roles

7. Layer 5: the ERPNext Stock module

"Stock" is one module of ERPNext (the same family as Buying, Selling and Accounts). PharmaStock uses only the Stock-related records below. It does **not** use ERPNext's Selling, HR or Accounts screens, though ERPNext requires a Company and a chart of accounts to exist.

ERPNext record	Used for	Key settings PharmaStock applies
Item	A medicine	Stock item, batch tracking on, expiry date on, shelf life 730 days, batch series `BATCH-.#####`, re-order level and quantity in the Pharmacy warehouse
Item Group	Medicine categories	A parent group "Medicine" with children: Analgesic, Antibiotic, Antimalarial, Antihypertensive, Antidiabetic, Antiseptic, Vitamin / Supplement, Vaccine, Gastrointestinal, Respiratory, Other Medicine
UOM	Units	Tablet, Capsule, Bottle, Vial, Sachet, Box, Tube, Ampoule, Strip, Pack (whole numbers only)
Warehouse	The pharmacy store	One warehouse named "Pharmacy" under "All Warehouses"
Batch	A delivery lot and its expiry date	Created on every `add_stock`
Stock Entry	Every movement	*Material Receipt* = stock in; *Material Issue* = stock out
Stock Ledger Entry	History and balances	Read directly with SQL for the dashboard, reports and movement history
Serial and Batch Bundle	Links a movement to its batches	Read to show which batch a movement used
Material Request	Stock requests	Type "Purchase". Submitted = approved. Rejected = our custom flag. ERPNext also creates these automatically when stock drops below the re-order level
Stock Settings	Site-wide rules	Batch tracking on, pick batches by **Expiry**, `auto_indent` (automatic re-order requests) on

What ERPNext does for us, so we do not have to

- Refuses a stock-out that would make the balance negative.
- Posts every movement into the Stock Ledger and keeps running balances.
- Tracks the quantity left in every batch.
- Creates re-order Material Requests when stock falls below a medicine's minimum (needs the scheduler running).
- Keeps documents immutable once submitted; cancelling reverses the ledger.

How we "talk to the stock module"

We do **not** call ERPNext over its web API. `api.py` runs in the same Python process and calls the framework directly, for example:

```
frappe.get_doc({
  "doctype": "Stock Entry", "stock_entry_type": "Material Issue",
  "from_warehouse": wh, "custom_reason": reason,
  "items": [{"item_code": medicine, "qty": q, "s_warehouse": wh, "batch_no": b}]
}).insert()   # then .submit()
```

On `submit()`, ERPNext's Stock Entry code runs its checks and writes the ledger. PharmaStock supplies the pharmacy rules around it (role check, earliest-valid-batch choice, idempotency).

> **Why not use ERPNext's own REST API?** It would need a second layer of authentication, one HTTP call per step (so a dropped connection could leave half a record), and pharmacy users would need ERPNext permissions. Direct calls give one atomic transaction and a smaller permission surface.

8. Layer 6: database, cache, workers

Service	Role
MariaDB	Every record (items, batches, entries, users). One site = one database.
Redis (cache)	Session and metadata cache.
Redis (queue)	Background-job queue, real-time events.
Workers (short, long)	Run queued jobs, such as sending notifications.
Scheduler	Runs periodic tasks: the "days before expiry" notification and automatic re-order. Without it those features silently do nothing.
gunicorn / nginx / websocket	Python web server, static files and reverse proxy, real-time channel.

9. Why PharmaStock has to be installed into ERPNext

This is the part that confuses most people, so here it is plainly.

- **Same process and same database.** Our API calls ERPNext's code as normal Python imports and writes to the same database. That is only possible if PharmaStock is *loaded by the same Frappe site*. It cannot run as a separate server that merely "talks to" ERPNext without going back to HTTP calls.
- **Installing the app is what sets everything up.** `install-app` runs our hooks, which create the roles, custom fields, categories, units, notifications, stock settings and the Pharmacy warehouse. Without the install none of those exist and every API call would fail.
- **It also registers the web route and the service worker.** `/pharmacy` only exists because the app's `hooks.py` is loaded by the site.

So what is the "desk"?

The **desk** is ERPNext's own administration interface at `/desk` (also called the Frappe Desk). "Installing the app in the desk" really means installing it **on the site**; the desk is where an administrator can see the app's tile (`add_to_apps_screen`) and inspect the underlying records.

	Pharmacy users	Administrator (IT)
Interface	`/pharmacy` (PharmaStock)	`/desk` (ERPNext) when needed
Roles	Pharmacy Staff / Pharmacy Admin	Administrator / System Manager
Desk access	None (`desk_access = 0`)	Full
Typical use	Receive, give out, request, view reports	First-time setup (company, accounts), backups, user and system settings

10. End-to-end flows

10.1 Log in and open the app

1. The user opens `/pharmacy`. Frappe returns the static shell.
2. The browser loads the JavaScript from `/assets/pharmastock/frontend/…` and registers the service worker from `/pharmacy/sw.js`.
3. The app calls `boot`. If there is no session it shows the login page; the login posts to Frappe's standard `/api/method/login`, which sets the session cookie.
4. `boot` is called again; it returns the user, role, options and a CSRF token. The dashboard loads.

10.2 Receive stock ("Add stock")

1. The user picks a medicine, quantity, expiry date, optional lot number and price.
2. The browser sends `add_stock` with a new `client_id`.
3. The API checks the role, rejects quantity ≤ 0 or an expiry not after the received date, and reuses an existing lot only if its item and expiry match.
4. It creates the **Batch**, then a submitted **Stock Entry (Material Receipt)** into the Pharmacy warehouse.
5. ERPNext posts the Stock Ledger entry. The dashboard figures change on the next read.

10.3 Give out stock

1. The user picks a medicine, quantity and a reason (dispensed, transfer, damaged, expired, other).
2. `_allocate` lists batches with stock, ordered by **earliest expiry**, and keeps only valid ones (or only expired ones when the reason is "Expired stock").
3. If the valid quantity is too small, the request is refused with a clear message (expired stock is never given to patients).
4. Otherwise one submitted **Stock Entry (Material Issue)** is created, with one row per batch used.
5. If the balance drops below the medicine's minimum, ERPNext's re-order rule creates a Material Request, and our notification tells the administrators.

10.4 Stock requests

1. Staff send `request_stock` (quantity, urgency Routine / Urgent / Emergency, reason). A draft **Material Request** is created, and administrators are notified.
2. An administrator calls `decide_request`. **Approve** submits the document; **reject** sets the custom "Rejected" flag and stores a note.
3. The app maps ERPNext's raw statuses (Draft, Pending, Ordered, Received…) to its own simple labels: pending, rejected, approved, ordered, received, cancelled.

10.5 Dashboard and reports

Read-only. Medicine status (ok, low, out, expiring, expired) is computed from item stock, batch expiry and the minimum level. The 14-day activity chart and the movement history are SQL queries on **Stock Ledger Entry** for the Pharmacy warehouse. Reports can be downloaded for Excel or printed.

10.6 Working offline

See section 3. In short: the service worker serves the app files, `localStorage` serves the last data, and writes wait in the outbox with their `client_id` until the server is reachable, then are sent exactly once.

11. What happens at install time

Hook	When	What it does
`after_install`	Right after `install-app`	Runs `after_migrate` ; then, only if ERPNext is ready (company exists), sets Stock Settings and creates the Pharmacy warehouse
`after_migrate`	Every update (`bench migrate`)	Adds anything missing: roles, custom fields, categories and units, notifications. Never overwrites settings an administrator changed
`setup_wizard_complete`	At the end of the ERPNext setup wizard	Finishes the setup if the app was installed *before* the company existed. A failure here is logged and no longer blocks the wizard

> **The warehouse and accounting.** ERPNext creates a company with *perpetual inventory* on. A warehouse can then only be created if the company has a **Default Inventory Account**. A French/OHADA chart of accounts has no account of type "Stock", so the warehouse failed. Choosing the **Standard** chart in the wizard solves it. Otherwise set a Default Inventory Account, or turn off perpetual inventory, then run `bench --site <site> execute pharmastock.setup.install.after_install` .

12. Deployment

12.1 Requirements

- Frappe and ERPNext **version 16** (`pyproject.toml` requires ≥16 and <17).
- About **2 to 4 GB of RAM** and a disk that persists. A site without these will be slow or lose data.
- A completed ERPNext setup wizard (a Company), with a chart of accounts that has a stock account (Standard).
- HTTPS, which the installable app and offline mode need.
- The scheduler enabled (for expiry alerts and automatic re-order).
- Backups. This is the only copy of the stock records.

12.2 The deployment options

Option	How	Notes
Frappe Cloud	Create a **private bench group** (v16), add the GitHub repo as a custom app, deploy, create a site, install ERPNext then PharmaStock	Managed backups, HTTPS and updates. Custom apps are not available on shared benches or the trial, and need a payment method and a paid plan. We checked and could not use the free tier for this reason
Docker (frappe_docker)	Use the `frappe/erpnext:v16` image plus an image with the app baked in, MariaDB, Redis, workers, scheduler, nginx	What we ran. You manage backups, updates and HTTPS yourself
Bare-metal bench	`bench get-app` then `bench --site … install-app pharmastock`	Classic method on a Linux server
Local network at the health center	Same Docker setup on a machine in the pharmacy	Free, fast, works without internet; HTTPS needs a local certificate for the installable/offline features

12.3 What we actually ran (Docker)

1. **Image.** A tiny Dockerfile starts from `frappe/erpnext:v16.52.1`, copies the app into `apps/pharmastock`, runs `pip install -e`, links `assets/pharmastock`, and adds the app to `sites/apps.txt`. This makes it survive container restarts (an earlier hand-copied version did not).
2. **Stack.** A compose file based on `frappe_docker`'s `pwd.yml` with: `db` (MariaDB 11.8), `redis-cache`, `redis-queue`, `configurator`, `create-site` (one-off), `backend` (gunicorn), `frontend` (nginx), `websocket`, `queue-short`, `queue-long`, `scheduler`, and `cloudflared`. Restart policy: `unless-stopped`. Database and admin passwords are generated and kept in a private `.env` file.
3. **Site creation.** `create-site` runs `bench new-site` with ERPNext, then `install-app pharmastock`.
4. **Public address.** A **Cloudflare Tunnel** connects the Docker network to Cloudflare, which gives HTTPS on `pharmastock.skyengpro.site` with no open ports on the machine. The tunnel's rule sends the hostname to `http://frontend:8080`. (The site is named `frontend` internally; nginx passes that name, so any hostname works.)
5. **First use.** Log in to `/app` as Administrator, run the setup wizard with the **Standard** chart, create a Pharmacy Admin and Pharmacy Staff user.

```
Browser ──HTTPS──► Cloudflare ──tunnel──► cloudflared ─► nginx (frontend:8080)
                                                        ├─► gunicorn (backend:8000) ─► MariaDB,
Redis
                                                        └─► websocket (9000)
workers + scheduler ─► Redis / MariaDB
```

12.4 Updating the app

1. Change the code, rebuild the front end if the UI changed (`npm run build` in `frontend/`), and commit **both** `pharmastock/public/frontend` and `pharmastock/www/pharmacy.html`.
2. Push to GitHub. On Frappe Cloud, deploy the bench. With Docker, rebuild the image and recreate the containers.
3. Run `bench --site <site> migrate` so `after_migrate` adds new fields without touching settings.

13. Problems we met, and the lessons

Problem	Cause	Fix / lesson
Setup wizard failed with "Please set Account in Warehouse Pharmacy…"	The wizard's hook created the warehouse before the company had an inventory account	Hook failure no longer blocks the wizard; use the Standard chart of accounts; run `after_install` afterwards if needed
`/pharmacy` blank page	nginx container did not contain the app's static files	Bake the app into the Docker image so every container has it
Site creation crashed at the last step	The app name was added to `apps.txt` in a volume shared with a container that lacked the app	Install the app with the supported command; keep all containers on the same image
Very slow pages	The host's internet connection (8-second ping, 25% loss). The app itself answered in 0.05 s locally	Host on a good connection, or on the local network. Cached files looked fast because Cloudflare served them
Dashboard showed zeros	The live site was empty, and the browser showed its cached empty copy while offline	Add data; reload when the connection is back
Free hosting	ERPNext needs several services and 2 GB+ of RAM; free tiers are too small or erase data	Local network, a small paid server, or Frappe Cloud paid

14. Operations checklist, and a file map

Before real use

- Delete the demo data, or start a fresh site, before entering real stock.
- Change every default or shared password (Administrator, pharmacy users).
- **Set up automatic backups** of the database and the `sites` folder, and test a restore.
- Confirm the scheduler is running (expiry alerts, re-order).
- Delete any API tokens used during setup (Cloudflare, Frappe).

- Check the host stays on, with a stable connection, and set restart-on-boot.

File map

Path	Contents
`pharmastock/hooks.py`	App registration, route rules, role home page, install hooks
`pharmastock/api.py`	The whole server API (layer 4)
`pharmastock/setup/install.py`	Roles, custom fields, catalogue, stock settings, warehouse, notifications
`pharmastock/service_worker.py`	Serves `/pharmacy/sw.js`
`pharmastock/www/pharmacy.html` / `.py`	The page shell (built)
`pharmastock/public/frontend/`	Built JavaScript, CSS and icons (committed)
`frontend/`	Vue 3 source: pages, API client, offline logic, translations
`tests/api_smoke.py`, `tests/e2e.mjs`	API checks and headless-Chrome journeys
`pyproject.toml`	Python package and the ERPNext/Frappe v16 requirement

This document describes the code as read from the repository on the date above. Detail such as exact status mappings and field lists may change as the app evolves; the source files named in each section are the reference.